

Exercise 3: Packet Inspection

Week: 3 (Hands-On Laboratory Exercise)

Objective: To design Node-RED endpoints that process URL parameters and queries, utilize HTTP headers for content typing and authentication, and utilize Wireshark to capture and inspect the raw HTTP transmission overhead over TCP/IP.

Step 1: Query Strings vs. Route Parameters in Node-RED

RESTful APIs allow clients to pass specific variables directly within the URL. We will build two new endpoints to compare Route Parameters (best for identifying a specific resource) and Query Strings (best for filtering or sorting).

1. Create a Route Parameter Endpoint:

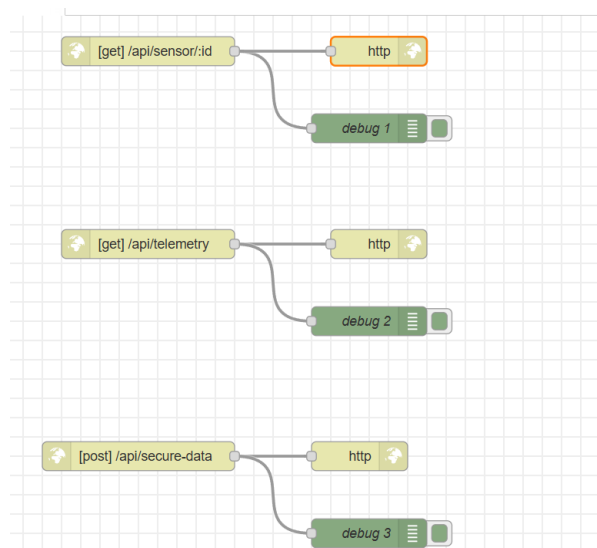
- Drag a new http in node onto the workspace.
- Set Method to GET.
- Set URL to `/api/sensor/:id` (The `:id` tells Node-RED to expect a dynamic variable).
- Wire it to a debug node (set to output complete msg object) and an http response node.

2. Create a Query String Endpoint:

- Drag another http in node onto the workspace.
- Set Method to GET.
- Set URL to `/api/telemetry` (No special characters needed; queries are appended automatically by the client).
- Wire it to a debug node and an http response node.

3. Deploy & Test in Postman:

- **Test 1 (Route):** Send a GET request to `http://localhost:1880/api/sensor/ESP32_Alpha`. Look at the Node-RED debug sidebar. Expand `msg.req.params` to see your ID successfully captured!
- **Test 2 (Query):** Send a GET request to `http://localhost:1880/api/telemetry?location=lab&status=active`. Look at the Node-RED debug sidebar. Expand `msg.req.query` to see your variables cleanly parsed into a JSON object!



Step 2: The Power of HTTP Headers (Media & Authentication)

Headers are the critical "metadata" of the HTTP request. They dictate how the server should parse the incoming byte stream and determine if the client even has permission to talk to the server.

1. Understanding Content-Type:

- An API can receive more than just text. The Content-Type header tells the server what the payload is.
- Common types include application/json (for our telemetry), image/jpeg (for an IoT camera feed), or audio/mpeg (for a microphone sensor).

2. Understanding Authorization:

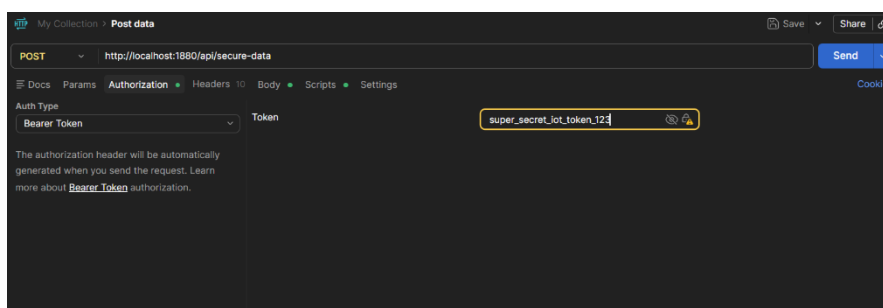
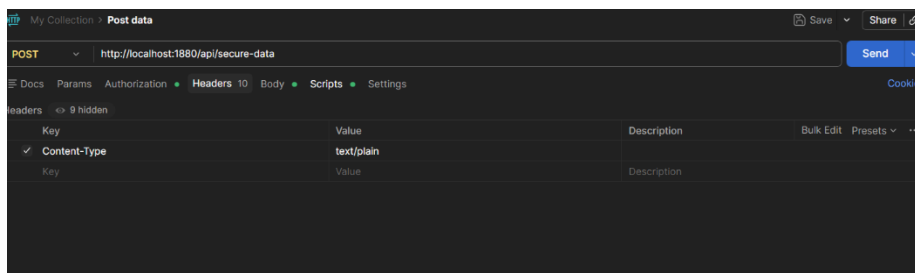
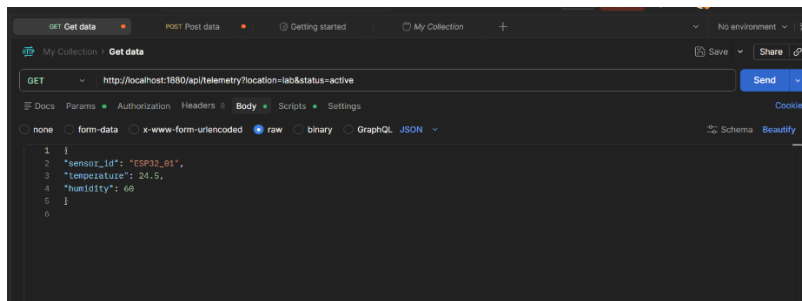
- APIs must be secured. The Authorization header passes credentials. Common types are Basic (Base64 encoded username/password) or Bearer (a secure token, like a JWT).

3. Configure Postman to Send Headers:

- In Postman, create a new POST request to `http://localhost:1880/api/secure-data`.
- Click the **Headers** tab. Add a new key Content-Type with the value `application/json`.
- Click the **Authorization** tab. Select Bearer Token and type a fake token:
`super_secret_iot_token_123`.
- Add a simple JSON body (e.g., `{"status": "secure test"}`).

4. Deploy a Catch-All in Node-RED:

- Create an http in node for POST `/api/secure-data`, wire to a response and debug node. Deploy.
- Send the Postman request. In the Node-RED debug window, expand `msg.req.headers`. You will see your authorization token and content-type clearly listed!



Step 3: Deep Packet Inspection with Wireshark

To truly visualize the theoretical efficiency (and immense overhead) of the protocols learned, students will utilize network protocol analyzers (Wireshark) to capture, filter, and inspect the raw network payloads.

1. Initialize Wireshark:

- Open Wireshark. Since we are testing locally via Docker, double-click the **Adapter for loopback traffic capture** (Windows) or **lo0** (macOS/Linux) interface to begin sniffing local traffic.

2. Apply a Capture Filter:

- The network is noisy. In the display filter bar at the top, type `tcp.port == 1880` and press Enter. Now you will only see traffic going to your Node-RED server.

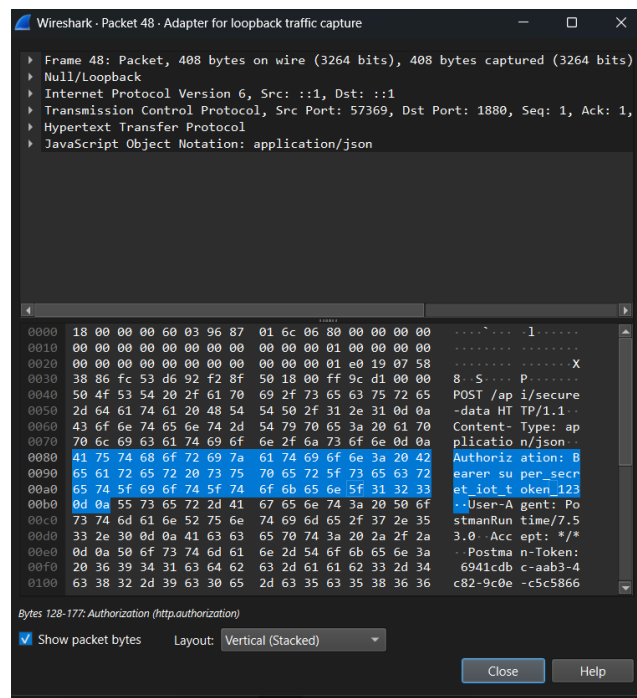
3. Trigger the Network Event:

- Go back to Postman and hit **Send** on your secure POST request from Step 2.

4. Analyze the Packet:

- In Wireshark, you will see a burst of TCP packets (the handshake) followed by an HTTP POST packet. Click on the HTTP POST packet.
- In the middle pane, expand the **Hypertext Transfer Protocol** section.
- Observation:** You can read the HTTP headers in plain text! You will clearly see `Authorization: Bearer super_secret_iot_token_123\r\n` and your JSON payload at the bottom. This proves why raw HTTP is dangerous for IoT and why TLS/HTTPS encryption is mandatory in production!

43	71.326447700	::1	::1	TCP	76	57369 → 1880 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
44	71.326553700	::1	::1	TCP	76	1880 → 57369 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
45	71.326594100	::1	::1	TCP	64	57369 → 1880 [ACK] Seq=1 Ack=1 Win=65280 Len=0
46	71.326732800	::1	::1	TCP	64	57368 → 1880 [FIN, ACK] Seq=1 Ack=1 Win=65280 Len=0
47	71.326757600	::1	::1	TCP	64	1880 → 57368 [ACK] Seq=1 Ack=2 Win=65280 Len=0
48	71.327035900	::1	::1	HTTP/1.1	408	POST /api/secure-data HTTP/1.1, JSON (application/json)
49	71.327059600	::1	::1	TCP	64	1880 → 57369 [ACK] Seq=1 Ack=345 Win=65024 Len=0
50	71.328354500	::1	::1	TCP	64	1880 → 57368 [FIN, ACK] Seq=1 Ack=2 Win=65280 Len=0
51	71.328394100	::1	::1	TCP	64	57368 → 1880 [ACK] Seq=2 Ack=2 Win=65280 Len=0
52	71.333699900	::1	::1	HTTP/1.1	305	HTTP/1.1 200 OK, JSON (application/json)
53	71.333756800	::1	::1	TCP	64	57369 → 1880 [ACK] Seq=345 Ack=302 Win=65024 Len=0
54	71.385876500	127.0.0.1	127.0.0.1	TCP	6715	1880 → 55174 [PSH, ACK] Seq=196 Ack=1 Win=252 Len=6671
55	71.385952300	127.0.0.1	127.0.0.1	TCP	44	55174 → 1880 [ACK] Seq=1 Ack=6867 Win=140 Len=0
56	72.350009100	::1	::1	TCP	65	[TCP Keep-Alive] 57369 → 1880 [ACK] Seq=344 Ack=302 Win=65024 Len=1
57	72.350056100	::1	::1	TCP	76	[TCP Keep-Alive ACK] 1880 → 57369 [ACK] Seq=302 Ack=345 Win=65024 Len=0 SLE=344 SRE=345
58	73.353537000	::1	::1	TCP	65	[TCP Keep-Alive] 57369 → 1880 [ACK] Seq=344 Ack=302 Win=65024 Len=1
59	73.353564000	::1	::1	TCP	76	[TCP Keep-Alive ACK] 1880 → 57369 [ACK] Seq=302 Ack=345 Win=65024 Len=0 SLE=344 SRE=345
60	74.355578000	::1	::1	TCP	65	[TCP Keep-Alive] 57369 → 1880 [ACK] Seq=344 Ack=302 Win=65024 Len=1
61	74.355651700	::1	::1	TCP	76	[TCP Keep-Alive ACK] 1880 → 57369 [ACK] Seq=302 Ack=345 Win=65024 Len=0 SLE=344 SRE=345
62	75.023176500	127.0.0.1	127.0.0.1	TCP	83	1880 → 55174 [PSH, ACK] Seq=6867 Ack=1 Win=252 Len=39



Part 4: Analytical Modifications & Network Comparison

Now that you have the tools, you must test the system's boundaries. Complete the following modifications and observe the changes in your Wireshark packet capture.

Exercise:

1. **Modify the Payload:** In Postman, change your JSON body from a short 20-character string to a massive block of text (simulating a large batch of sensor data). Send the request.
2. **Modify the Header:** Change your Content-Type header to text/plain and send it again.
3. **Analyze the Delta in Wireshark:**
 - Locate the two new HTTP packets in Wireshark.
 - Look at the Length column for the massive payload. How many bytes did your transmission size increase by?
 - Expand the HTTP section for the second packet. How did the Content-Type header change visually in the raw byte stream?

